

Waarom ik voor recursie koos:

Voor dit onderdeel heb ik ervoor gekozen om zowel de maze generator als de path finder recursief te implementeren. Hierdoor gebruiken beide algoritmes dezelfde denkwijze en sluiten ze inhoudelijk goed op elkaar aan.

Dat gaf twee voordelen:

- ❖ De logica blijft relatief compact en overzichtelijk.
- ❖ De visualisatie werd veel duidelijker, omdat je letterlijk ziet hoe het algoritme stap voor stap de diepte in gaat.

Door dezelfde recursieve aanpak te gebruiken, wordt het bovendien eenvoudiger om de werking van beide algoritmes met elkaar te vergelijken.

Hoe mijn recursive path finder werkt:

De path finder start bij het beginpunt van de maze en onderzoekt vervolgens recursief alle geldige buurcellen. Zodra het eindpunt wordt bereikt, stopt het algoritme direct met zoeken.

Een belangrijk onderdeel van mijn implementatie is het bijhouden van **parent-relaties**. Hiermee wordt opgeslagen vanaf welke positie een cel is bereikt. Wanneer het eindpunt gevonden is, kan ik daardoor achteraf het volledige correcte pad reconstrueren.

Voorbeeld van de stopconditie:

```
if (current[0] == maze.End[0] && current[1] == maze.End[1])  
{  
    found = true;  
    endPosition = current;  
    return;  
}
```

Tijdens het zoeken doe ik ook markeringen voor visualisatie:

- ❖ 4 voor verkend;
- ❖ 5 voor fout spoor (backtracking).

Daardoor zie je niet alleen de finale route, maar ook alle mislukte vertakkingen die het algoritme eerst probeerde.

Voorbeeld van parent-registratie en verkenning:

```
if (cellValue == 0 || cellValue == 2)
{
    parent[newKey] = current;
    visitedPositions.Enqueue(newPos);

    if (maze.MazeArray[newRow][newCol] != 2)
        maze.MazeArray[newRow][newCol] = 4;

    Recurse(newPos);

    if (!found)
    {
        maze.MazeArray[newRow][newCol] = 5;
        visitedPositions.Enqueue(newPos);
    }
}
```

Deze aanpak zorgt ervoor dat het volledige zoekproces zichtbaar blijft en maakt het eenvoudiger om de werking van het algoritme te analyseren.

Hoe mijn recursive maze generator werkt:

Voor de generator gebruik ik depth-first carving met sprongen van 2 cellen. Ik begin op (1,1), kies een willekeurige richting, breek de tussenmuur open en gaat de recursie dan verder.

In plaats van één cel tegelijk te bewegen, wordt steeds twee cellen vooruitgekeken. Hierdoor blijven muren behouden tussen de paden. Wanneer een nieuwe cel wordt

gekozen, wordt de tussenliggende muur verwijderd en gaat het algoritme recursief verder vanuit de nieuwe positie. Deze aanpak zorgt voor een natuurlijke doolhof.

Voorbeeld van de carve-stap:

```
grid[row + dRow / 2][col + dCol / 2] = 0;
```

```
CarvePassagesRecursive(grid, visited, nextRow, nextCol);
```

Na het genereren van de maze wordt het eindpunt niet willekeurig geplaatst. In plaats daarvan gebruik ik een **Breadth-First Search (BFS)** om vanaf het startpunt het verste bereikbare punt te vinden.

Hierdoor wordt de afstand tussen start en eindpunt meestal groter, wat resulteert in een interessantere en uitdagendere maze.

Voorbeeld van de BFS-uitbreiding:

```
distance[nextRow][nextCol] = currentDistance + 1;
```

```
queue.Enqueue(new int[] { nextRow, nextCol });
```

Deze methode levert doorgaans een langere oplossingsroute op dan wanneer het eindpunt willekeurig wordt gekozen.

Wat ik sterk vind aan mijn eigen aanpak:

- ❖ De generator en solver passen inhoudelijk bij elkaar door dezelfde recursieve stijl.
- ❖ De visualisatie laat echt het denkproces zien, niet alleen het eindantwoord.
- ❖ Door het verste eindpunt te kiezen wordt de maze merkbaar uitdagender.
- ❖ De structuur laat toe om later eerlijk te vergelijken met Dijkstra, A* en stack-based oplossingen.

Wat ik nog kan verbeteren:

- ❖ [Een iteratieve fallback met een expliciete stack zou dit robuuster maken.
- ❖ Ik kan extra metrics toevoegen, zoals bezochte knopen, totale tijd en pad lengte, zodat de vergelijking tussen algoritmes objectiever wordt.

Conclusie:

Door zowel de maze generator als de path finder recursief op te bouwen, heb ik een consistente en overzichtelijke oplossing ontwikkeld. De combinatie van recursie, visualisatie en het gebruik van BFS voor het bepalen van een geschikt eindpunt zorgt voor een goed inzicht in de werking van de algoritmes. Daarnaast biedt de huidige structuur een sterke basis voor toekomstige uitbreidingen en vergelijkingen met andere zoekalgoritmes.